Mongod process : Manages the database
Mongo shell : Database management & Querying

* MongoDB :-

Date: / /

- MongoDB is a NoSQL, Document-oriented database.
- It stores data in type of JSON format called BSON.
- Allowing for more efficient & dynamic modelling.
- Record in MongoDB is a document.
  It is a data structure composed of key value pair similar to structure of JSON object.
- It works on concept of collection & document.
- Single MongoDB server has multiple interface & database.
- Collection is group of document equivalent to RDBMS table.
- field values may include numbers, strings, booleans, arrays or even nested documents.
- File format used for MongoDB configuration : "YAML"

• Advantages of MongoDB over RDBMS :-

- Schema less
- Structure of object is clear.
- No complex join
- Optimized for read & write operations in big data scenarios.
- Supports horizontal scaling through "sharding"
- Supports secondary indexes for faster query execution.
- Replication : Data availability & reliability through replica sets.
- Rich Query language : Query operalitions like filter, projection, sorting & joins (via lookup)

Collection : MongoDB :: Table : RDBMS

MongoDB not supports iOS

Default port for MongoDB : 27017
Default document size limit : 16 MB Date: / /

- Geospatial Queries : Allows querying on location based data.
- Integrates with various programming langauges

Use cases :
- Content management systems
- IOT applications
- Real-time Analytics
- Mobile & web applications
- Social networks.

* Operating systems supported by MongoDB
  - Windows
  - Linux
  - MacOS

- Example Document :

```
{
  title : "Post title",
  body : "Body of post",
  category : "News",
  likes : 1,
  tags : ["news", "events"],
  date : Date()
}
```

* Basic MongoDB Commands :
  - connect to database -
    1. mongo
    2. by using mongodb shell : mongosh

  - start mongodb :
    sudo service mongodb start

  - Stop MongoDB :
    sudo service mongodb stop

db.users.update({},{})

*update() by default replace the data of entire document

Date: / /

• Restart MongoDB :

sudo service mongodb restart

• show databases

show dbs

• Create / Use database :

use database-name (e.g. use newdb)

• create collection :

db.createcollection ("collection_name")

• Insert document : insert single document

db.collection_name.insertOne({key: "value"})

• insert multiple documents

db.collection_name.insertMany([{key1: "value1"},
{key2: "value2"}])

• Find documents :

db.collection_name.find()

db.collection_name.find({key: "value"})

• Update Document : (update single document which matches filter)

db.collection_name.updateOne({key: "value"},
{$set: {key: "new_value"}})

db.collection_name.updateMany({key: "value"},
{$set: {key: "new_value"}})

* The fields which are not specified in set part they remain unchanged.

* $unset operator is used to delete a field from document.

MongoDB handles schema validation through user-defined validation rules.
Use references instead of embedding when data duplication is concern.

Date: / /

- Delete Document :

db.collection_name.deleteOne({key: "value"})

db.collection_name.deleteMany({key: "value"})

- Drop collection :

db.collection_name.drop()

- Drop Database :

db.dropDatabase()

* If you have not selected any database, then it will delete default 'test' database
[test database is by default created in MongoDB]

- Create collection :

db.createCollection("Name", option)

⇒ Name - String type - name of the collection to be created

⇒ option - Document type - Specify options about memory size & indexing
(It is optional)

- show collections : show collections

* You can also pass array of document into insert method

*• db.name.save() ⇒ It is also used to insert
If you specify _id of already existing document then it will replace whole data of document containing _id specified

Date: / /

* MongoDB Query API :-

It is the way you will react with your data.

MongoDB query API can be used in two ways :

- ① CRUD Operations
- ② Aggregation Pipelines

- It is used to perform :
  - Adhoc queries with mongosh, Compass, VS Code or MongoDB driver for programming language you use.
  - Data transformations using aggregation pipelines
  - Document join to combine data from different collections
  - Graph & geospatial queries
  - Full text search
  - Indexing
  - Time series Analysis

* In MongoDB, a database is not actually created until in gets content. [Insert something into database to see it in list]
- Same for collection

* If you try to insert documents into a collection that does not exist, MongoDB will create the collection automatically.

Date: / /

MongoDB Datatypes :

String : mostly used to store data
String MongoDB must be UIF-8 valid

Integer : store numeric data
32 bit or 64 bit, depend on server

Boolean : store boolean values

Double : store floating point values

Min/Max Keys : used to compare a value against lowest & highest BSON elements

Arrays : store arrays or list or multiple values in one key.

Timestamp : timestamp. This can be handy for recording when a document has been modified or added.

Object : used for embedded document

Null : used to store null value

Symbol : used identically for languages that use a specific symbol type.

Date : used to store the current date or time in UNIX time format.
Specify your own time by creating object of Date & passing day, month, year onto it.

Object ID - used to store document ID.

Date: / /

Binary data : store binary data
Code : used to store javascript code into document.
Regular Expression : used to store regular expression

* If we don't assign object ID to document then (i.e. -id parameter not mention at the time of insert) MongoDB assign unique object ID for the document

* _id is 12 bytes hexadecimal number unique for each document in collection.
12 bytes divided as,
4 bytes timestamp
3 bytes machine id
2 bytes process id
3 bytes incrementer

* • find() ⇒ To select/query data from MongoDB collection
- This method accepts a query object. If left empty all documents will be returned.
- It will display all the documents in non-structured way.
• pretty() : used to display the results in formatted way.

db.collection_name.find().pretty()

The find() method in MongoDB returns a cursor to the documents that match the query criteria.

Date: / /

* **Where clause :-**

- RDBMS where clause equivalents in MongoDB.
- Used to query the document on the basis of some condition.

① Operation - equality.

syntax : { key : { $eq : "value" } }

e.g. : db.mycol.find ({"by" : "tutorial point"}).pretty()

RDBMS equivalent : where by = "tutorial point"

- Not equal : $ne

  e.g. {"status" : { $ne : "active" } }

② less than : { key : { $lt : value } }

  e.g. db.mycol.find({"price" : { $lt : 500 }})

③ greater than : { key : { $gt : value } }

  { "key": { $gte : value } } greater than equal

④ in : { "key" : { $in : ["value1", "value2"] } }

  Matches value that exist in array.

⑤ not in : $nin

  Values not in an array.

* These conditional operators are used for string data also.

Date: / /

* Logical operators:

① AND : $and (Returns documents where both queries match)

syntax : db.collection_name.find({$and:[{key1:value1}, {key2:value2}]})

② OR : $or (Either query matches)

Syntax :
```
db.collection_name.find(
    {
        $or:[
        {key1:value1}, {key2:value2}
        ]
    }
).pretty()
```

③ AND and OR together

④ Nor : $nor (Both queries fail to match)

{$nor:[{key1:value1}, {key2:value2}]}

⑤ Not : $not (where query does not match)

not used with in, gt, lt, gte, lte, neq, eq

* Evaluation:

① $regex : Allows the use of regular expression when evaluating field values

② $text : Performs a text search

③ $where : Use a Javascript expression to match documents

Date: / /

Update Method : update()

Update method updates the value in existing document.

Syntax : db.collection_name.update()

- Update Methods :

1. $currenDate : Sets the field value to the current date
2. $inc : Increament the field value
3. $rename : Renames the field
4. $set : Set the value of a field
5. $unset : Removes the field from the document

- Methods for updating array :

1. $addToSet : Add distinct elements to an array
2. $pop : Removes the first or last element of an array
3. $pull : Removes all elements from array that match the query
4. $push : Adds an element to an array

- findOneAndUpdate()

- This methods updates the values in existing document
- First find one document then update

- remove() : It is also used to delete document from collection

- It works on 2 parameters i.e. deletion criteria & just one flag

db.collection.remove(query, justOne)

justOne : If set to true, then it removes only 1 document where match is found. (By default false ; remove all where query matches)

db.collection.remove ({status : "inactive"}, true)

- This deprecated in newer version of MongoDB

Date: / /

• Aggregation Pipelines :

① $group : group documents by specific field and perform operations (like sum, avg etc) on grouped data

$group is deprecated in favour of aggregate

```
db.collection.group( {
    key: { },
    cond : {condn},
    reduce : function(curr, result) { },
    initial : { },
    finalize: function(result) { } #optional
    } ) ;
```

```
db.collection.aggregate ([
  {
    $group : {
      _id : "$category",                    // Group by category field
      totalQuantity : {$sum : "$quantity"}  // sum of quantity field
    }
  }
]);
```

② $limit : It limits the number of document passed to the next stage

To limit the records in MongoDB

This method accepts one number type argument

```
db.collection.aggregate ([{$limit : 1}])
```

Or

```
db.collection.find().limit(number)
```

If don't specify the number argument in limit the it will display all documents.

Date: / /

③ $project : It is used to shape or transform the documents by adding, including or excluding new fields

- which field include or exclude in the output.
- Compute new fields using expressions.
- Rename fields.

Main use:

- Field selection : include / exclude
- Field Transformation : create / modify fields
- Field renaming
- Control output : Hide or include required data

```
db.collection.aggregate([
  {
    $project : {
      field1 : 1          # include field1
      field2 : 0          # exclude field2
      newField : { $sum : ["$field3", "field4"] }   # create computed field
    }
  } , {limit 5 }          # it is optional
])
```

_id field is included by default. It is included unless specifically excluded.

④ $sort : This is used to sort documents

1 is for ascending order & -1 is for descending order

```
db.collection.aggregate([ { $sort : { "field" : -1 } },
                          { "Field" : 1,
                            "field1" : 0 }
                        ])
```

$match: This is a filtering operations.
Gives the documents where matches found
It can reduce the amount of documents
Using this method early in the pipeline, it can improve performance.

```
db.collection.aggregate([
    {$match: {$property_type: "House"}}, ...
])
```

⑥ $addFields: It is used to add new field

```
db.collection.aggregate([
    { $addFields: {
        field name ← avgGrade: { $avg: "$grades.score" }
                    }   avg method
    },
])
```

⑦ $count: It will return the count of specific fields
Return the number of documents that match a specified query in collection

#without filter: db.collection.count()
return total number of documents

#with filter: db.collection.count({field: value})
count documents that match specific criteria

#using aggregate:

```
db.collection.aggregate([ { $match: { "cuisine": "chinese" } },
                          {$count: "total chinese" }
                        ])
```

Date: / /

⑧ $lookup : This stage performs a left outer join to a collection in the same database

In this, there are 4 required fields,

1] from : The collection to use for lookup in the same database

2] localField : The field in the primary collection that can be used as a unique identifier in the from collection

3] foreignField : The field in the from collection that can be used as a unique identifier in the primary collection

4] as : The name of the new field that will contain the matching documents from the from collection.

from : Name of the collection to join with
localField : Field in the local collection to match
foreignField : Field in the foreign collection to match
as : Name of the field in the output document that stores the result of the join.

```
db.collection.aggregate([
  {
    $lookup : {
      from : "customers",              // Foreign collection : customers
      localField : "customer_id",      // Field in collection
      foreignField : "_id",            // Field in customers
      as : "customerDetails"           // output array field
    }
  }
]);
```

Date: / /

⑨ $out : It is used to write result of the aggregation pipeline to a new or existing collection
- This stage must be the last stage of aggregation pipeline

* MongoDB Drivers :
MongoDB has many language drivers
Following are the current officially supported drivers:
- C
- C++
- C#
- Go
- Java
- Python
- Node.js
- PHP
- Ruby
- Rust
- Scala
- Swift.